Weight Compensation Design

Project: Baby Milk Tracker / MilkWise
Author: Kit
Date: 2026-06-15
Status: Analysis — not yet implemented

---

1. Problem Statement

The daily milk target is computed as:

```
dailyTargetMl = weightKg × mlPerKgPerDay
```

The weight used in this formula is a static value stored in `settings.weightKg`. As the baby gains weight her nutritional need grows, but the target stays fixed unless a parent manually updates it in Settings.

The consequence is systematic error. A baby who weighed 5.0 kg at birth and now weighs 6.5 kg has 30% higher caloric needs. If the target was never updated, the app underestimates her true need. Every status number (smoothed %, strict 24h %, next feed time, best bottle size) will then trend toward "overfed" even when she is actually on track. Parents will hold back feeds unnecessarily.

The solution is to use historical weight measurements to estimate the baby's weight at any past or future moment, and to derive the effective target from that estimate rather than from a frozen setting.

Key constraint: All derived numbers must remain derived. Nothing new is stored in the database as a result of this feature. The weight history (already stored in `weights.json`) is the only input.

---

2. Current Data Available

Live instance (idea02): 11 weight entries spanning 2026-04-20 to 2026-06-15, from 4.13 kg to 6.93 kg. 143 feed entries spanning 2026-05-31 to 2026-06-15. This is excellent Jenss-Bayley territory — nearly two months of weight history covering the critical deceleration phase.

```
[
  { "id": "mq74acizfy", "timestamp": 1776804780000, "weightKg":
4.21 },
```

```
  { "id": "mq74iipx01", "timestamp": 1776718740000, "weightKg":
4.13 },
  { "id": "mq7496ny3t", "timestamp": 1777582320000, "weightKg":
4.80 },
  ...
  { "id": "mqer7medtd", "timestamp": 1781500140000, "weightKg":
6.93 }
]
```

The app already has a `weightAtTime(t, weights, fallback)` utility that returns the most recent weight entry at or before time `t`. This is used by the trend graph. The weight compensation scheme builds on and replaces this with a proper growth model.

---

3. Pediatric Growth Models

Three models are relevant. We evaluate each for practical fit with available data.

3.1 WHO Child Growth Standards (LMS method)

What it is:
The WHO published Child Growth Standards in 2006, derived from a longitudinal study of healthy term infants in six countries (Brazil, Ghana, India, Norway, Oman, United States) raised under optimal conditions. The standards define weight-for-age reference distributions separately for boys and girls, at weekly intervals from birth to 5 years.

The LMS parameterisation:
Each data point in the WHO tables is described by three parameters — L (Box-Cox power), M (median weight in kg), and S (coefficient of variation). Together they define the entire distribution of weights at that age:

```
Z = [ (X/M)^L − 1 ] / (L × S)
```

where `X` is the child's observed weight and `Z` is the resulting WHO z-score (standard deviations from the median).

What a z-score means:
A z-score of 0 means the child is exactly at the median for her age and sex. A z-score of +1 means she is 1 standard deviation above the median (roughly the 84th percentile). A z-score of −1 means she is roughly the 16th percentile. Healthy children typically remain on their individual growth channel — their z-score stays approximately constant over months.

How you would use it for prediction:

1. Calculate the baby's z-score from each weight measurement: `Z = [(X/M)^L − 1] / (L × S)`
2. Average the z-scores across all measurements (or use the most recent few). This gives you her individual growth channel.
3. To predict her weight at a future age `t`, look up `M(t)`, `L(t)`, `S(t)` from the WHO table and invert the formula: `X(t) = M(t) × (1 + L(t) × S(t) × Z_mean)^(1/L(t))`

Strengths:
Clinically validated on a diverse global population. Statistically rigorous. Captures the known non-linear shape of infant growth (fast early, slowing after 3 months). Naturally handles sex differences.

Weaknesses for our use case:
Requires knowing the baby's exact age in days and sex to look up the LMS table. Requires embedding the full WHO table in the app (manageable, ~200 rows per sex). The z-score approach assumes the child is healthy and growing along her channel — it will not detect illness-related weight loss without special handling. For healthy term infants tracked weekly, this is the gold-standard approach if we want maximum scientific rigour.

When to choose this:
If we have the baby's date of birth and sex stored, and we want the most clinically grounded prediction with automatic deceleration as the baby ages past 3 months.

3.2 Jenss-Bayley Model

What it is:
The Jenss-Bayley (1937, extended by Bock et al. 1973) model is a parametric growth curve for infancy:

```
W(t) = a + b·t − exp(c + d·t)
```

where `t` is age in months and `W(t)` is weight in kg. The parameters `a`, `b`, `c`, `d` are fitted to the individual's measurement history. The exponential term `exp(c + d·t)` captures the characteristic rapidly-decelerating postnatal weight gain. Because `d` is negative, the exponential term shrinks as `t` increases.

What each term does:

- `a + b·t` — linear trend component; describes the long-run steady growth rate
- `−exp(c + d·t)` — the initial rapid burst of growth above the linear trend; starts large and shrinks exponentially

At birth (t=0), the formula equals `a + b·0 − exp(c)`. As `t → ∞`, `exp(c + d·t) → 0` and the curve asymptotes to the linear trend `a + b·t`. This means the formula is effectively linear once the exponential term is negligible.

When does the exponential term become unimportant?
The exponential term becomes negligible when `exp(c + d·t)` is small relative to the overall weight. This depends on the fitted `d` parameter, but typical values from published studies (Jenss & Bayley 1937, Bock et al. 1973, Berkey & Reed 1987) show the transition at approximately **3–4 months of age**. Beyond 4 months, the Jenss-Bayley curve is nearly indistinguishable from a straight line. This aligns with the well-known clinical observation that infants double their birth weight by around 4–5 months and then grow more steadily.

Practical implication: If the baby is already past ~4 months, the exponential term contributes very little and the model degenerates to linear growth. For babies in the 0–12 week window, the exponential term matters significantly.

Fitting the model:
Jenss-Bayley requires non-linear least squares fitting (e.g. Levenberg-Marquardt). With 27+ feed records and several weight measurements, we have more than enough data. In fact, the parameters are typically stable with as few as 6–8 weight measurements spanning at least 4–6 weeks.

Why we can use this:
With the feed logs going back some weeks and weight measurements at irregular intervals, we have the data density needed for a reliable fit. The model is specifically designed for infancy (first 2 years) and accounts for the biological shape of growth rather than assuming it is linear. Published reference parameters are available (e.g. from the Fels Longitudinal Study) and can serve as starting priors or sanity bounds.

Weaknesses:
More complex to implement than linear regression. Requires a non-linear solver (implementable in TypeScript with a simple gradient descent or bisection, but not trivial). Needs careful initialisation to avoid converging to a wrong local minimum. If the baby has fewer than 4–5 weight measurements, the exponential and linear terms are not independently identifiable.

When to choose this:
If the baby is young (0–4 months), if we want biological accuracy, and if we have at least 5–6 weight measurements spanning several weeks.

3.3 Linear Weighted Least Squares (Practical Baseline)

What it is:
Fit a straight line `W(t) = a + b·t` through all weight measurements using

ordinary or weighted least squares. Extrapolate to the current day to get `predictedWeightKg`.

Strengths:
Trivial to implement. Numerically stable. Interpretable (b = grams/day growth rate). No risk of over-fitting. Sufficient accuracy over short windows (1–3 weeks). Already used informally by paediatricians ("she's gaining 25g per day, on track").

Weaknesses:
Cannot capture the deceleration that occurs after 3 months. If you fit a line through weights at 2 months and 5 months, the line will underestimate how fast she was growing at 2 months and overestimate how fast she is growing at 5 months. Over multi-month windows, this matters.

WHO velocity sanity bounds (use as a clamp regardless of model):

Age range	Expected weight gain (median)
0–1 month	26–31 g/day
1–2 months	24–28 g/day
2–3 months	18–23 g/day
3–6 months	12–18 g/day
6–9 months	8–12 g/day
9–12 months	6–9 g/day

Any extrapolated growth rate outside ±50% of these bounds for the baby's age should be clamped, to protect against noise in sparse measurements.

---

4. Model Recommendation

Given that: - We already have 27+ feed log entries spanning multiple weeks - We have at least several weight measurements (more will accumulate) - The implementation complexity needs to be manageable

Recommended approach: Jenss-Bayley with linear fallback.

- If fewer than 4 weight measurements: use linear weighted least squares.
- If 4+ measurements and age < 6 months: use Jenss-Bayley (most biologically accurate for the exponential growth phase).
- If age > 6 months: Jenss-Bayley degenerates to near-linear anyway; either model is equivalent.
- Always apply WHO velocity sanity bounds as a safety clamp on the extrapolated growth rate.

Future option: If we store date of birth and sex, add the WHO LMS z-score model as the third-tier predictor for maximum clinical accuracy. This can be introduced later without changing the architecture.

---

5. Proposed Architecture

5.0 Onboarding gate

`weights.json` must never be empty at runtime. Instead of a fallback to `settings.weightKg`, the app shows an onboarding screen when `weights.length === 0`:

- **Trigger:** Dashboard `load()` finds `weights.length === 0` → redirect to `/onboarding`
- **Onboarding collects:** current weight (kg) + date/time of measurement (defaults to now)
- **On submit:** POST to `/api/weights` → creates first `WeightEntry` → redirect to dashboard
- **Result:** The dashboard is never rendered without at least one weight entry

This makes `settings.weightKg` redundant as a calculation input. It remains in `settings.json` only for legacy compatibility and is kept in sync by the weights POST route, but no calculation code reads it as the authoritative weight.

5.1 New function: `predictWeightKg(weights, atTime)`

```
// lib/weightGrowth.ts (new file)

export function predictWeightKg(
  weights: WeightEntry[],
  atTime: number           // target time (ms)
): number
// Precondition: weights.length >= 1 (guaranteed by onboarding)
```

Algorithm:

1. Sort weights by timestamp ascending.
2. If 1 entry: return that entry's `weightKg` (no extrapolation — not enough data).
3. If 2–3 entries: linear weighted least squares (recent measurements weighted more heavily).
4. If 4+ entries: attempt Jenss-Bayley fit; fall back to linear if fit fails to converge.

5. Clamp extrapolated growth rate against WHO velocity bounds for estimated age.
6. Return predicted weight at `atTime`.

Note: the `fallback` parameter is removed entirely. Onboarding guarantees the precondition.

5.2 Effective weight at time T

The existing `weightAtTime(t, weights, fallback)` function in `lib/weights.ts` currently uses step-function interpolation (last observed weight). This will be replaced or augmented:

```
// Replaces weightAtTime for forward-looking use
export function effectiveWeightAtTime(
  t: number,
  weights: WeightEntry[],
  fallback: number
): number {
  if (weights.length === 0) return fallback;

  // For times within the observation window: linear interpolation
between measurements
  // For times after the last measurement: model-based
extrapolation
  // For times before the first measurement: earliest recorded
weight
  ...
}
```

Critical design constraint: The function remains pure — it takes `weights` as input and returns a number. Nothing is written to disk.

5.2b Removal of `targetMlPerDay` from feeds

`Feed.targetMlPerDay` is a stored derived value — it encodes what `weightKg × mlPerKgPerDay` was at log time. This is now fully derivable from `weights.json`:

```
// Before (reading stored stamp)
const target = feed.targetMlPerDay ?? currentTargetMl;

// After (pure derivation)
```

```
const target = dailyTargetAtTime(feed.timestamp, weights,
mlPerKgPerDay);
```

Migration plan: - Stop writing `targetMlPerDay` in the feed POST route immediately - Remove `targetMlPerDay` from the `Feed` type (make reads fall through to derived value) - Remove `migrateTargetStamps()` — no longer needed - Update `dailyTotals()` to accept `weights: WeightEntry[]` and call `dailyTargetAtTime()` - Old entries with the field stored: ignore the field, use derived value instead

5.3 Daily weight update (start of day)

Rather than updating continuously (which would cause the target to drift upward through the day, which feels odd), the effective weight is snapped to a day boundary:

```
const startOfToday = new Date();
startOfToday.setHours(0, 0, 0, 0);
const effectiveWeight = predictWeightKg(weights,
startOfToday.getTime());
```

This means the target is stable throughout the day and only steps up at midnight when you cross into a new day. This is consistent with the milkwise display principle: meaningful snapshots, not continuous drift.

5.4 Impact on displayed data

Every value that currently uses `settings.weightKg` will instead use `effectiveWeight` derived from the growth model:

- `dailyTargetMl = effectiveWeight × settings.mlPerKgPerDay`
- `hourlyRate = dailyTargetMl / 24`
- All percentages, next feed times, bottle size recommendations — all derived from these two, automatically correct.

The subtitle on the dashboard currently shows `6.93 kg · Target: 1040 ml/day`. With this change it would show:

```
6.95 kg (estimated) · Target: 1043 ml/day
```

The `(estimated)` label is shown whenever the weight is model-predicted (i.e. `weights.length >= 2` and we are beyond the last measurement date). When the reference time is between two known measurements, the label shows `(interpolated)`.

5.5 Nothing new is stored

The predicted weight is computed on every page load from the raw `weights.json` entries. No new file, no new field, no new API endpoint. The computation is fast (< 1ms for any realistic number of weight entries) and adds no observable latency.

This also means `targetMlPerDay` is no longer stamped on new feeds (§5.2b). The only fields written per feed remain `id`, `timestamp`, and `volume` — the three immutable facts.

---

6. Display Considerations

- **Dashboard subtitle:** Show effective weight with `(estimated)` when using model prediction.
- **Analytics page (Weight tab):** Add a dashed extrapolation line beyond the last measurement, showing the model's prediction for the next 7 days.
- **Settings page:** Remove the editable `weightKg` field when weight history is present (or keep it as a manual override). The model supersedes it.
- **History page (Weight tab):** No change — already shows all weight entries.

---

7. Open Questions

1. **Date of birth:** Should we collect date of birth and sex during onboarding? This enables proper age-based WHO velocity bounds and eventual WHO LMS z-score support. Storage: two optional fields in `settings.json` (`dateOfBirthMs`, `sex: 'M'|'F'`). These are genuine primary facts (not derived) so storing them is correct.

2. **Settings weight field:** With onboarding in place, the Settings weight field becomes misleading. Options: (a) remove it — weight is managed via the ⚖ button or onboarding only; (b) keep it but have it create a `WeightEntry` rather than update `settings.weightKg` directly. Option (a) is cleaner.

3. **Minimum measurement gap:** A Jenss-Bayley fit needs measurements spread over at least 3–4 weeks to independently estimate the parameters. If all 4+ measurements are from the same week, fall back to linear.

4. **Retrospective correction:** Historical status numbers (Analytics trend, daily totals) could also be corrected by using weight-at-time. The `trendGraph.ts buildTrendPoints` function already does this — each feed point uses `dailyTargetAtTime()`. So retrospective

correction is already partially in place; it just needs `dailyTargetAtTime` to use the model rather than the step function.

---

8. Implementation Steps (when approved)

Phase 1 — Data hygiene (no visible UI change) 1. Remove `targetMlPerDay` write from feeds POST route. 2. Update `dailyTotals()` to accept `weights[]` and use `dailyTargetAtTime()` — remove `targetMlPerDay` read. 3. Remove `migrateTargetStamps()`. 4. Remove `targetMlPerDay` from `Feed` type (keep readable for old data, never write).

Phase 2 — Onboarding 5. Create `/onboarding` page: weight input + date/time + submit → POST `/api/weights` → redirect to dashboard. 6. Add onboarding gate to dashboard `load()`: if `weights.length === 0` → `router.push('/onboarding')`. 7. Optionally collect date of birth and sex during onboarding for future WHO LMS support.

Phase 3 — Growth model 8. Create `lib/weightGrowth.ts` with `predictWeightKg(weights, atTime)` — linear fit first. 9. Update `lib/weights.ts`: `effectiveWeightAtTime()` calls `predictWeightKg`. 10. Update `page.tsx` dashboard: compute `effectiveWeight` at start-of-today, derive `dailyTargetMl` and `hourlyRate` from it. 11. Update dashboard subtitle to show estimated/interpolated weight with label. 12. Update `trendGraph.ts` to use the model for historical target lines.

Phase 4 — Jenss-Bayley 13. Add non-linear solver to `weightGrowth.ts`; switch to Jenss-Bayley when `weights.length >= 4`. 14. Add extrapolation projection line to Analytics Weight chart. 15. Write unit tests: one weight (identity), linear fit (2–3 points), Jenss-Bayley (4+ points), WHO clamp.